Trial Autopsy: how the sponsor technology is used

How Zetaris, Meterless and NVIDIA are used, and what is distinctive about each. Every claim points to the code and to a way to check it. The numbers come from the 41 recorded analyses (36 blind backtests and 5 examples; the sixth example is the ACCELERATE backtest itself). Every model answer was recorded live through Nightshift and the NVIDIA API on 24 and 25 October 2026. Their logs are in `logs/`.

Team CurrentSky · Stanislav Gatin (solo) Repository: `docs/SPONSOR_TECHNOLOGY.md`, `metadata.json`

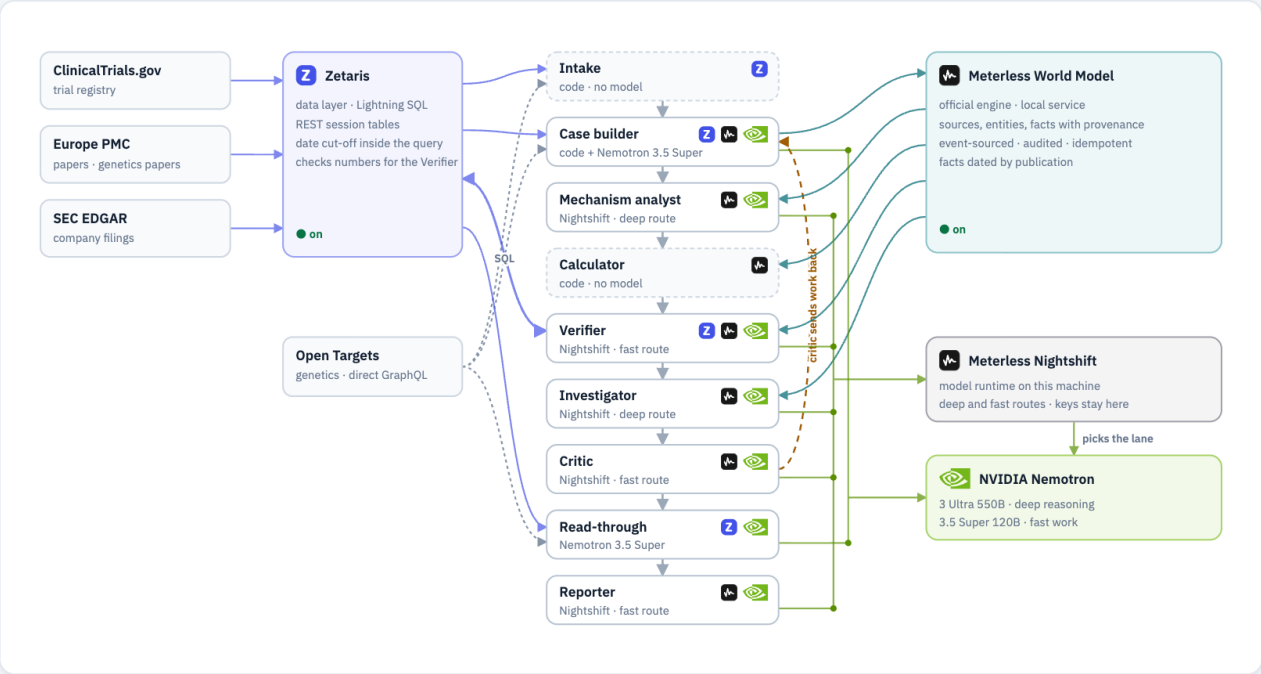
At a glance

Sponsor	Role in the product	Code

Zetaris	Data layer: the trial registry, the literature and company filings are read through Zetaris with SQL, and the date cut-off runs inside the query. The Verifier has Nemotron write the SQL that checks every number the calculator uses against its source.	`app/tools/zetaris.py`
		`app/tools/http.py`
		`app/tools/europepmc.py`
		`app/agents/verifier.py`

Meterless	Two roles. **Power:** Meterless Nightshift is the primary model provider. **Build:** the official World Model engine stores every source, entity and fact with its provenance.	`app/llm/meterless_runtime.py`
		`app/llm/generator.py`
		`services/meterless/`
		`app/engines/world_model.py`

NVIDIA	Every AI step runs on NVIDIA Nemotron (3 Ultra and 3.5 Super).	`app/config.py` `app/llm/generator.py`
Cursor	Development: the unit tests were written in it, and its Agent searched the codebase for bugs and gaps.	`tests/`



The system map from the app's Settings page: data sources and Zetaris on the left, the nine steps in the middle, the Meterless World Model, Meterless Nightshift and NVIDIA Nemotron on the right. Sponsor logos mark each step that uses that sponsor's technology.

Zetaris: the data layer

What it does. Trial Autopsy reads ClinicalTrials.gov, Europe PMC and SEC EDGAR through a Zetaris instance instead of calling those APIs directly. Each request is registered as a Zetaris REST session table and read back with Lightning SQL over the Zetaris REST API:

```
CREATE REST SESSION TABLE ta_<id> REQUEST (endpoint "<source URL>", method "GET", ...)
  HEADER (user-agent "...") BODY ()
SELECT to_json(r) FROM session.ta_<id> LATERAL VIEW explode(resultList.result) AS r
  WHERE r.firstPublicationDate <= '<as_of>'
```

Why it matters here. A blind analysis may only see documents published before the trial's result. For the literature, Zetaris applies that cut-off inside the SQL query, so a paper from after the cut-off is filtered out by the data layer before any agent sees it.

Resilience. If Zetaris is unreachable, the read falls back to the direct request and the run trace records a warning. Open Targets (a GraphQL POST API) is read directly.

The Verifier: Nemotron writes SQL, Zetaris checks the numbers

After the calculator has run, the Verifier takes each number it used (a biomarker change, planned events, follow-up) and checks it against its source through Zetaris:

1. The registry record and the cited papers are registered as Zetaris REST session tables (aliases `registry` and `papers`).
2. Nemotron writes one Lightning SQL query per number, for example:

   ```
   SELECT regexp_extract(r.abstractText, '([^.]*993[^.]*)', 1) FROM papers
     LATERAL VIEW explode(resultList.result) AS r WHERE r.pmid = '40185318'
   ```

3. Code accepts only one SELECT over the known tables, with no comments or other statements, and adds a row limit. In a blind backtest it also refuses fields known only after the readout: posted results, status, actual enrolment.

4. Zetaris runs the query. If Zetaris returns an error, Nemotron gets one chance to fix the SQL.
5. Nemotron judges each number from the rows that came back: confirmed, not found or contradicts. Code adds one rule: a number with no rows is never confirmed.

A contradiction goes to the critic as a serious issue, and the critic can send the work back (up to two rounds). Without Zetaris, the report says its numbers were not independently verified and lowers its confidence one level.

Across the 41 analyses the Verifier checked 146 numbers: 99 confirmed, 46 not found in the abstract or registry, 1 contradiction, which reached the critic. In dal-OUTCOMES "2.5 years" of follow-up meant that 80% of patients were followed for at least 2.5 years, not the median; the critic flagged it as a serious issue and sent the work back. In pelacarsen the Verifier found that the 2.5-year follow-up is a minimum, and the analyst had already used the midpoint of the minimum and the planned duration, 3.25 years.

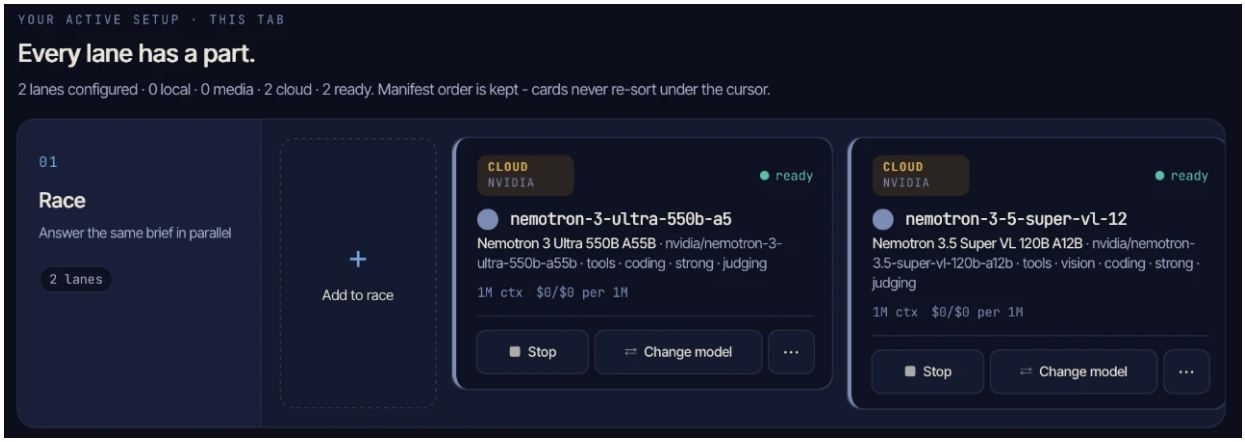
> HOW TO CHECK
>
> - Settings page → Zetaris card → *Test connection*; `scripts/smoke_test.py` runs `SELECT 1` over the REST API.
> - A live run's step log shows `read via Zetaris`, `register_tables`, `write_sql` and `query_zetaris` lines.
> - The report footer says `data layer: Zetaris (N source reads via Lightning SQL)`; the *Numbers used* tab shows every query and verdict.
> - `tests/test_verifier.py` covers the SQL guard, the cut-off rule, the run without Zetaris and a contradiction reaching the critic.

Meterless Nightshift: the model runtime (Power)

What it does. Nightshift is the app's primary AI provider, following the Meterless Runtime provider contract (`app/llm/meterless_runtime.py`):

- it calls Nightshift's OpenAI-compatible endpoint (default `http://127.0.0.1:8088/v1`) with the runtime token from Nightshift's Console;
- every request carries the header `meterless-app: trial-autopsy`, and chat requests also carry `meterless-scope: user`;
- each step asks for a routing mode, chosen per step on the Settings page from Nightshift's `GET /v1/models` list (`meterless/deep`, `meterless/fast`, `meterless/auto`); Nightshift picks the Nemotron lane that answers;
- Settings has Auto Detect (ports 8088, 8090, 8787, 11434), paste of a connect link, and a connection test that reads `/v1/models` and sends one request.

The provider keys stay in Nightshift. Each response's `model` field names the lane that answered, and the trace records it. Answers are stored under the model that actually answered, so one lane's answer is never replayed as another model's.



Nightshift's lanes: Nemotron 3 Ultra and Nemotron 3.5 Super VL on its NVIDIA provider, both ready.

Through Nightshift in the recorded runs. Nightshift served the five reasoning steps: the mechanism analyst, the Verifier, the investigator, the critic and the reporter. In the 41 recorded runs 373 of the 377 calls went

through Nightshift; the other 4 went to NVIDIA directly after Nightshift cut them twice. 361 answers came from Nemotron 3 Ultra and 16 from Nemotron 3.5 Super. By the amount of model work Nightshift is the main route: 2.54 million of the 3.6 million tokens (71%) and 76% of the generated tokens; by count, more calls go straight to NVIDIA (572 of 945), because extraction reads each document in many short calls with reasoning off. A full live analysis took a median of 13 minutes (5 to 37). Per-run table: `docs/evidence/nightshift_runs.md`.

What I learned running a pipeline through Nightshift

Nightshift is built first for chat. A pipeline of strict steps needs a few settings, and I measured each:

What I found	Effect on the pipeline	How the app handles it
Build 0.92.0 hid NVIDIA's models in the lane picker (`nvidia` missing from `byokProviders`)	Nemotron could not be added to a lane	Reported with the one-line fix (Meterless/Meterless#4); fixed in 0.92.1 the same night
Tier-split, on by default, adds a planner phase before the request	Strict extraction rules were diluted: one document gave 9 facts directly and 0 through Nightshift	Tier-split off in *Setup → Preferences*
The reasoning switch (`chat_template_kwargs`) is not passed on	With reasoning on, extraction found 0 to 3 facts where reasoning off found 9	Extraction and read-through call NVIDIA's API directly with reasoning off
For an identified app, the mode bound in the Console decides the routing	One routing mode for the whole app	The app is bound to `fast`; Nightshift picks the lane
A cloud request with no first token after 45 s is cut	Long reasoning prompts on a busy Nemotron 3 Ultra were cut	Streaming, a watchdog, retries, and a direct NVIDIA call after two cuts (needed 0 times in the final runs, 5 times in earlier ones). I reported it; Meterless is making the limit configurable
An answer cache keyed on the last 4,000 characters of the prompt	One run got the previous run's report back; a repeated connection test was answered from the cache	Every request carries a unique reference, so the cache cannot replay a stale answer

> HOW TO CHECK
>
> - Settings page → Meterless card → *Auto Detect*, then *Test connection*: the routing modes Nightshift serves and the lane that answered.
> - `scripts/smoke_test.py`: "Nemotron replies through Meterless Nightshift" passes only if a Nemotron model answered.
> - In a run's step log, every reasoning step shows `meterless:nvidia/nemotron-...`.

Meterless World Model: the evidence store (Build)

What it does. The official Meterless World Model reference engine (from github.com/Meterless/Meterless, unchanged, Apache-2.0) runs on the machine as a local service in `services/meterless/`. `run.py` starts it, and the Docker image includes it. Every source, entity and fact the agents produce is written to it, one namespace per run, with provenance: the step that found it; the run; the source URL; the source's publication date, used as the observation time.

Facts whose quote failed the word-for-word check are stored too, marked unverified with low confidence; only verified facts reach the calculator and the report. The engine keeps an event-sourced, audited, idempotent world state. `app/engines/world_model.py` implements the same three services the Meterless engine guides describe (WorldStore, IngestPipeline, ContextBuilder), and the analyst, the calculator, the Verifier and the investigator build their context from them. Across the 41 analyses the engine stored 869 entities, 2,983 facts, 845 links and 7,440 audit entries; each run's audit log is `logs/world-<run_id>.jsonl`.

HOW TO CHECK

- The official conformance suite passes 8 of 8 checks:
  `npm –prefix services/meterless run conformance`
- Settings page → Meterless World Model → *Test connection*; `scripts/smoke_test.py` checks the service.
- Each run's step log ends with a `world_model_sync` line, for pelacarsen: 20 entities · 62 facts · 20 links · 157 audit entries.

NVIDIA Nemotron: the models

What it does. Every AI step runs on NVIDIA Nemotron.

Step	Model in the recorded runs	Route	Reasoning
Mechanism analyst: picks the numbers the calculator uses	Nemotron 3 Ultra (550B)	Nightshift	on
Verifier: writes the SQL that checks each number in Zetaris	Nemotron 3 Ultra (550B)	Nightshift	on
Investigator: weighs every possible cause	Nemotron 3 Ultra (550B)	Nightshift	on
Critic: checks the work and can send it back	Nemotron 3 Ultra (550B)	Nightshift	on
Reporter	Nemotron 3 Ultra (550B)	Nightshift	on*
Extraction: copies numbers with verbatim quotes	Nemotron 3.5 Super (120B)	NVIDIA API	off
Read-through: rival drugs on the same target	Nemotron 3.5 Super (120B)	NVIDIA API	off

*Requested off; Nightshift does not pass the switch on. Nightshift picks the lane for each request; with the app bound to `fast`, it served the reasoning steps almost entirely from Ultra.

Engineering for Nemotron in a strict pipeline

`app/llm/generator.py`

- **The reasoning switch per role.** `chat_template_kwargs.enable_thinking` is set for each step: on where the model must think, off where it must copy exactly. On the same document, extraction with reasoning off found 9 facts in seconds, with reasoning on 0 to 3 in minutes.
- **Reasoning kept out of the answer.** Nemotron's reasoning arrives as `<think>`; code keeps only the answer, even when a proxy drops the opening tag.
- **Broken JSON repaired by the same model.** A reply cut off mid-thought gets one repair pass by the same model, or is asked once more.
- **Long calls that cannot hang.** Reasoning calls stream; a watchdog closes a silent stream, and after two cut-offs the call goes to NVIDIA directly (4 of 377 reasoning calls), named in the step log.
- **One rate limit shared.** Nightshift calls NVIDIA with the same key, so every call, through either route, is paced under one limit.
- **The steps check each other.** The critic has its own instructions and reviews the analyst's work; the Verifier checks the numbers against the sources.

Code does the checks the model could get wrong

Numbers come from verified quotes. Code then sets the direction of a change from the source's own words (reading the start of the sentence when a quote begins mid-sentence), converts units, and fills planned events or a baseline from cited facts. The run trace records each such step.

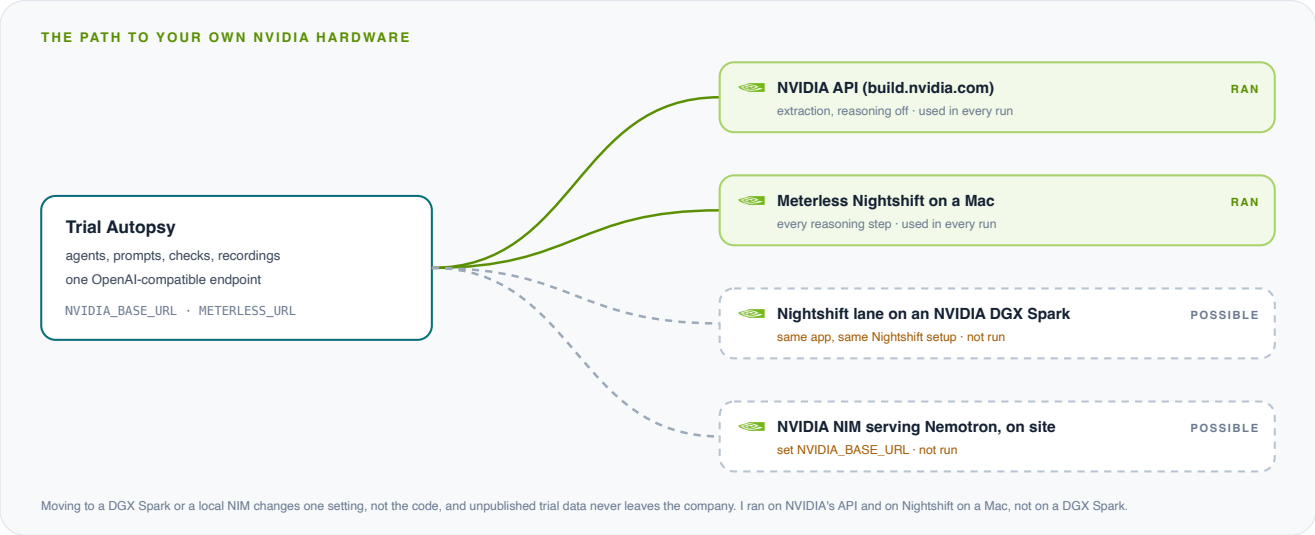
Measured

36 finished trials, replayed blind with only the documents published before each result, scored against the explanations experts published later (`eval/results/backtest_2026-10-25.md`). Six of them were added after the analysis code was frozen, and nothing was changed because of their results (`backtest_2026-10-25_frozen6.md`):

Blind backtest	All 36	6 after the freeze
Experts' cause as the main cause	**26 / 36**	**4 / 6**
Success or failure forecast right, where the trial could be sized	**16 / 22**	**1 / 1**
Expected hazard ratio within 0.03 of the answer key	8 / 25	0 / 1
Softer: one of the causes named matches one of the experts' causes	35 / 36	6 / 6

The calculator's probability of success has a Brier score of 0.137 on the 18 trials it could size (0.250 for always saying 50%); all 9 trials it gave below 20% or above 80% went the way it said.

Every recorded run replays offline with the same outcome, expected hazard ratio, power and fact count as the live run (`scripts/check_replay.py`).



Every model call goes to one OpenAI-compatible endpoint: moving to a DGX Spark or an on-site NIM is one setting.

Why open models. A drug company cannot send unpublished trial data to a cloud AI it does not control. Because every step runs on open Nemotron models, the same analysis can run on the company's own hardware, for example an NVIDIA DGX Spark, which Nightshift supports. In Trial Autopsy the switch is one setting, the model address; the agents, prompts and checks stay the same. I ran on NVIDIA's API and on Nightshift on a Mac; I have not run the system on a DGX Spark.

> HOW TO CHECK
>
> - Settings page: every AI step shows the NVIDIA logo and its Nemotron route or model.
> - Each report's agent graph and *What this analysis used* panel show the model each step actually used.
> - `eval/results/` holds the scores; `logs/` holds every recorded run's step log; `scripts/check_replay.py` replays each run offline and checks it reproduces the live result.

What is distinctive

Zetaris enforces the time machine.
A blind analysis cannot leak future information: SQL removes later papers before any agent sees them.

Zetaris checks the numbers.
Nemotron writes the SQL, code guards it, Zetaris runs it, and a contradiction goes to the critic. In dal-OUTCOMES that loop showed that “2.5 years” of follow-up was a minimum for 80% of patients, not the median.

Meterless in both required roles.
373 of the 377 reasoning calls in the recorded runs went through Nightshift; the World Model engine runs unchanged and passes its conformance suite. I reported a Nightshift bug with its fix (Meterless/Meterless#4), fixed the same night, reported the first-token limit, which Meterless is making configurable, and documented the settings a strict pipeline needs.

Open models, measured.
The whole pipeline runs on open NVIDIA models and is scored on blind backtests against expert explanations, with every recorded run reproducible offline.

Trial Autopsy · Made by CurrentSky · Research tool, not medical or investment advice. NVIDIA and Nemotron are trademarks of NVIDIA Corporation; Zetaris and Meterless of their owners.